
## German International University - GIU

Faculty of Informatics and Computer Science

CNET101: Computer Networks

# GIU-Net: UDP Chat Application & Campus Network Simulation

Spring 2026 Project Report


| Full Name | Student ID |
|---|---|
| Carlos Emad | 19001366 |
| Mark Tamer | 19008287 |
| Ahmed Roshdy | 19008541 |
| Omar Shaker | 19010121 |


Instructor: Dr. Maggie Shammaa

Submission Date: June 1, 2026

# 1. Team Information

## 1.1 Team Members & Derived Values

| Full Name | Student ID |
|---|---|
| Carlos Emad | 19001366 |
| Mark Tamer | 19008287 |
| Ahmed Roshdy | 19008541 |
| Omar Shaker | 19010121 |

## 1.2 Port Number Derivation

Team Leader: 19001366 (lowest student ID in the team)

Last 4 digits of Leader ID: 1366

Since 1366 >= 1024, no adjustment needed.

**Server Port = 1366**

## 1.3 Handshake String Derivation

The Sum is calculated from the last digit of each team member's student ID:

| Student ID | Last Digit | Contribution |
|---|---|---|
| 19001366 | 6 | 6 |
| 19008287 | 7 | 7 |
| 19008541 | 1 | 1 |
| 19010121 | 1 | 1 |
| Total Sum | | 15 |

**Client sends: HELLO-GIU-19001366-15**

**Server replies: OK-GIU-19001366-15**

# 2. Protocol Design Choices

## 2.1 Retransmission Timeout Value

We chose a retransmission timeout of 3 seconds as specified by the project requirements. This value was appropriate for our testing environment: both machines were connected to the same Wi-Fi network with round-trip times well under 100ms, meaning 3 seconds provides a generous window to account for any temporary congestion or packet processing delay. In practice, all our ACKs arrived within 50ms, so the timeout was never triggered during normal operation — only during our deliberate fault-injection tests.

The timeout was implemented using Java's LinkedBlockingQueue.poll(3000, TimeUnit.MILLISECONDS). This method blocks the sender thread for up to 3 seconds waiting for an ACK to be placed into the queue by the receiver thread. If no ACK arrives within the window, poll() returns null, which we treat as a timeout and trigger the retransmission logic.

## 2.2 Thread Coordination

Each side (server and client) runs two threads simultaneously:

- The main thread reads from the keyboard (stdin) and sends DATA packets.
- A background ReceiverThread continuously listens for incoming packets.

The key challenge was coordinating ACK delivery between threads. When the receiver thread receives an ACK packet, it parses the sequence number and puts it into a shared LinkedBlockingQueue<Integer>. The sender thread, which is blocked on queue.poll() waiting for the ACK, wakes up immediately when the receiver deposits the value.

This design avoids busy-waiting and race conditions. The queue is thread-safe by design (it is from java.util.concurrent), so no explicit synchronization was needed. The sender and receiver never write to the same variable simultaneously — the queue acts as the only shared communication channel between them.

While the sender thread is blocked on `ackQueue.poll(3000, MILLISECONDS)` waiting for an ACK, the receiver thread independently continues reading from the socket — the two threads never block each other because the keyboard read and the socket receive run on completely separate threads with no shared locks.

# 3. Debugging Story

## 3.1 The Bug: Handshake Always Rejected

Symptom: Every time the client sent the HELLO packet, the server replied with REJECTED, even though we were certain the handshake string was correct. The client printed:

```
[CLIENT] Server replied: REJECTED
```

Initial hypothesis: We initially thought the port number derivation was wrong, or that the server was receiving packets from a different client by mistake.

Diagnosis using Wireshark: We opened Wireshark and captured traffic on udp.port == 1366. In the packet's Data field, we could read the raw payload as plain UTF-8 text. The HELLO packet showed:

```
HELLO-GIU-19001366-15
```

Notice the trailing space. When we printed the received payload on the server side, it looked identical to the expected string. However, after adding a length check, we found the received payload was 22 characters while the expected string was 21. The extra character was a trailing space caused by how we were constructing the string with an accidental concatenation.

The fix: We added .trim() when reading the payload on the server side, and also fixed the string construction on the client. After the fix, the handshake completed successfully on the first attempt.

Lesson learned: Wireshark was essential here. Without seeing the raw bytes, we would never have noticed the invisible trailing space. This experience demonstrated exactly why capturing live traffic is a valuable debugging tool.

# 4. VLSM Subnet Table

## 4.1 Base Network Derivation

Team Leader ID: 19001366

Digit pair extraction: 19. 00. 13. 0

Note: The /22 block is aligned to the nearest /22 boundary below 19.0.13.0, which is 19.0.12.0.

**Base Network: 19.0.12.0/22**

Address range: 19.0.12.0 – 19.0.15.255 (1024 total addresses, 1022 usable hosts)

19. 0. 12. 0

00010011.00000000.00001100.00000000

/22 mask:

11111111.11111111.11111100.00000000

Network bits (22): 00010011.00000000.000011

Host bits (10): 00.00000000

Address range: 19.0.12.0 – 19.0.15.255

Total addresses: 2^10 = 1024 | Usable: 1022

## 4.2 VLSM Calculations (Largest First)

**Subnet C — Student Housing (200 hosts required)**

Step 1 — Find block size:

Hosts needed: 200

Add network + broadcast: + 2

Minimum block size: 202

Next power of 2 ≥ 202:

2^7 = 128 → too small

2^8 = 256 → fits

Host bits = 8

Prefix = 32 − 8 = /24

Block size = 256 addresses

Step 2 — Network address in binary:

Starting from base: 19.0.12.0
19.0.12.0: 00010011.00000000.00001100.00000000
|_____**network (24 bits)**_____|  |_*host(8)*_|

Step 3 — Subnet mask in binary:

/24: 11111111.11111111.11111111.00000000 = 255.255.255.0
|_____***24 ones***_____| |***8 zeros***|

Step 4 — Derive addresses:

Network : 00010011.00000000.00001100.00000000 = 19.0.12.0

First IP : 00010011.00000000.00001100.00000001 = 19.0.12.1

Last IP : 00010011.00000000.00001100.11111110 = 19.0.12.254

Broadcast : 00010011.00000000.00001100.11111111 = 19.0.12.255

Usable hosts: 2^8 − 2 = 254 satisfies 200

Next available block starts at: 19.0.13.0

Need >= 202 addresses (200 hosts + network + broadcast)

Next power of 2: 256 = 2^8 => /24 prefix

| Field | Value |
|---|---|
| Network Address | 19.0.12.0 |
| Subnet Mask | 255.255.255.0 |
| Prefix Length | /24 |
| First Usable IP | 19.0.12.1 |
| Last Usable IP | 19.0.12.254 |
| Broadcast | 19.0.12.255 |
| Usable Hosts | 254 (satisfies 200 requirement) |

Next available block starts at: 19.0.13.0

**Subnet B — Student Lab (100 hosts required)**

Step 1 — Find block size:

Hosts needed: 100

Add network + broadcast: + 2

Minimum block size: 102

Next power of 2 ≥ 102:

2^6 = 64 → too small

2^7 = 128 → fits

Host bits = 7

Prefix = 32 − 7 = /25

Block size = 128 addresses

Step 2 — Network address in binary:

Continuing from: 19.0.13.0
19.0.13.0:
00010011.00000000.00001101.00000000
|_____**network (25 bits)**_____| *host (7)* |

Step 3 — Subnet mask in binary:
/25: 11111111.11111111.11111111.10000000 = 255.255.255.128
|_____***25 ones***_____| |**7 zeros**|

Step 4 — Derive addresses:

Network : 00010011.00000000.00001101.00000000 = 19.0.13.0

First IP : 00010011.00000000.00001101.00000001 = 19.0.13.1

Last IP : 00010011.00000000.00001101.01111110 = 19.0.13.126

Broadcast : 00010011.00000000.00001101.01111111 = 19.0.13.127

Usable hosts: 2^7 − 2 = 126 satisfies 100

Next available block starts at: 19.0.13.128

Need >= 102 addresses. Next power of 2: 128 = 2^7 => /25 prefix

| Field | Value |
|---|---|
| Network Address | 19.0.13.0 |
| Subnet Mask | 255.255.255.128 |
| Prefix Length | /25 |
| First Usable IP | 19.0.13.1 |
| Last Usable IP | 19.0.13.126 |
| Broadcast | 19.0.13.127 |
| Usable Hosts | 126 (satisfies 100 requirement) |

Next available block starts at: 19.0.13.128

**Subnet A — Staff Offices (50 hosts required)**

Step 1 — Find block size:

Hosts needed: 50

Add network + broadcast: + 2

Minimum block size: 52

Next power of 2 ≥ 52:

2^5 = 32 → too small

2^6 = 64 → fits

Host bits = 6

Prefix = 32 − 6 = /26

Block size = 64 addresses

Step 2 — Network address in binary:

Continuing from: 19.0.13.128
19.0.13.128:
00010011.00000000.00001101.10000000
|_____**network (26 bits)**_____| |host(6)|

Step 3 — Subnet mask in binary:
/26: 11111111.11111111.11111111.11000000 = 255.255.255.192
|_____***26 ones***_____| |*6 zeros*|
Step 4 — Derive addresses:
Network : 00010011.00000000.00001101.10000000 = 19.0.13.128
First IP : 00010011.00000000.00001101.10000001 = 19.0.13.129

Last IP : 00010011.00000000.00001101.10111110 = 19.0.13.190
Broadcast : 00010011.00000000.00001101.10111111 = 19.0.13.191
Usable hosts: 2^6 − 2 = 62 ✓ satisfies 50

Next available block starts at: 19.0.13.192

Need >= 52 addresses. Next power of 2: 64 = 2^6 => /26 prefix

| Field | Value |
|---|---|
| Network Address | 19.0.13.128 |
| Subnet Mask | 255.255.255.192 |
| Prefix Length | /26 |
| First Usable IP | 19.0.13.129 |
| Last Usable IP | 19.0.13.190 |
| Broadcast | 19.0.13.191 |
| Usable Hosts | 62 (satisfies 50 requirement) |

Next available block starts at: 19.0.13.192

**Subnet E — Server DMZ (10 hosts required)**

Step 1 — Find block size:

Hosts needed: 10

Add network + broadcast: + 2

Minimum block size: 12

Next power of 2 ≥ 12:

2^3 = 8 → too small

2^4 = 16 → fits

Host bits = 4

Prefix = 32 − 4 = /28

Block size = 16 addresses

Step 2 — Network address in binary:

Continuing from: 19.0.13.192
19.0.13.192: 00010011.00000000.00001101.11000000
|_____**network (28 bits)**_____|host|

Step 3 — Subnet mask in binary:
/28: 11111111.11111111.11111111.11110000 = 255.255.255.240
|_____***28 ones***_____||4 zeros|

Step 4 — Derive addresses:
Network : 00010011.00000000.00001101.11000000 = 19.0.13.192
First IP : 00010011.00000000.00001101.11000001 = 19.0.13.193
Last IP : 00010011.00000000.00001101.11001110 = 19.0.13.206
Broadcast : 00010011.00000000.00001101.11001111 = 19.0.13.207
Usable hosts: 2^4 − 2 = 14 satisfies 10

Next available block starts at: 19.0.13.208

Need >= 12 addresses. Next power of 2: 16 = 2^4 => /28 prefix

| Field | Value |
|---|---|
| Network Address | 19.0.13.192 |
| Subnet Mask | 255.255.255.240 |
| Prefix Length | /28 |
| First Usable IP | 19.0.13.193 |
| Last Usable IP | 19.0.13.206 |
| Broadcast | 19.0.13.207 |
| Usable Hosts | 14 (satisfies 10 requirement) |

Next available block starts at: 19.0.13.208

**Subnet D — WAN Link (2 hosts required)**

Step 1 — Find block size:

Hosts needed: 2

Add network + broadcast: + 2

Minimum block size: 4

Next power of 2 ≥ 4:

2^2 = 4 → fits exactly

Host bits = 2

Prefix = 32 − 2 = /30

Block size = 4 addresses

Step 2 — Network address in binary:

Continuing from: 19.0.13.208
19.0.13.208: 00010011.00000000.00001101.11010000
|_____**network (30 bits)**_____| |host|

Step 3 — Subnet mask in binary:

/30: 11111111.11111111.11111111.11111100 = 255.255.255.252
|_____***30 ones***_____| |2 zeroes|

Step 4 — Derive addresses:
Network : 00010011.00000000.00001101.11010000 = 19.0.13.208
First IP : 00010011.00000000.00001101.11010001 = 19.0.13.209 (R1 WAN)
Last IP : 00010011.00000000.00001101.11010010 = 19.0.13.210 (R2 WAN)
Broadcast : 00010011.00000000.00001101.11010011 = 19.0.13.211
Usable hosts: 2^2 − 2 = 2 exactly satisfies requirement

Need exactly 4 addresses (2 hosts + network + broadcast). 4 = 2^2 => /30 prefix

| Field | Value |
|---|---|
| Network Address | 19.0.13.208 |
| Subnet Mask | 255.255.255.252 |
| Prefix Length | /30 |
| First Usable IP | 19.0.13.209 (Router 1 WAN interface) |
| Last Usable IP | 19.0.13.210 (Router 2 WAN interface) |
| Broadcast | 19.0.13.211 |
| Usable Hosts | 2 (exactly satisfies requirement) |

## 4.3 Summary Table

| Subnet | Purpose | Network | Mask | Prefix | First IP | Last IP | Broadcast | Device Assignment |
|---|---|---|---|---|---|---|---|---|
| C | Housing | 19.0.12.0 | 255.255.255.0 | /24 | 19.0.12.1 | 19.0.12.254 | 19.0.12.255 | R2 Fa0/0=.1, PCs=.2–.5 |
| B | Lab | 19.0.13.0 | 255.255.255.128 | /25 | 19.0.13.1 | 19.0.13.126 | 19.0.13.127 | R1 Fa0/0=.1, PCs=.2–.5 |
| A | Staff | 19.0.13.128 | 255.255.255.192 | /26 | 19.0.13.129 | 19.0.13.190 | 19.0.13.191 | R1 Fa0/1=.129, PCs=.130–.131 |
| E | DMZ | 19.0.13.192 | 255.255.255.240 | /28 | 19.0.13.193 | 19.0.13.206 | 19.0.13.207 | R1 Fa1/0=.193, Web-Srv=.194, GIU-Srv=.195 |
| D | WAN | 19.0.13.208 | 255.255.255.252 | /30 | 19.0.13.209 | 19.0.13.210 | 19.0.13.211 | R1 Se0/3/0=.209, R2 |

| Subnet | Purpose | Network | Mask | Prefix | First IP | Last IP | Broadcast | Device Assignment |
|---|---|---|---|---|---|---|---|---|
| | | | | | | | | Se0/3/0=.210 |

# 5. Routing Tables

## 5.1 Router 1 (Router0) Routing Table

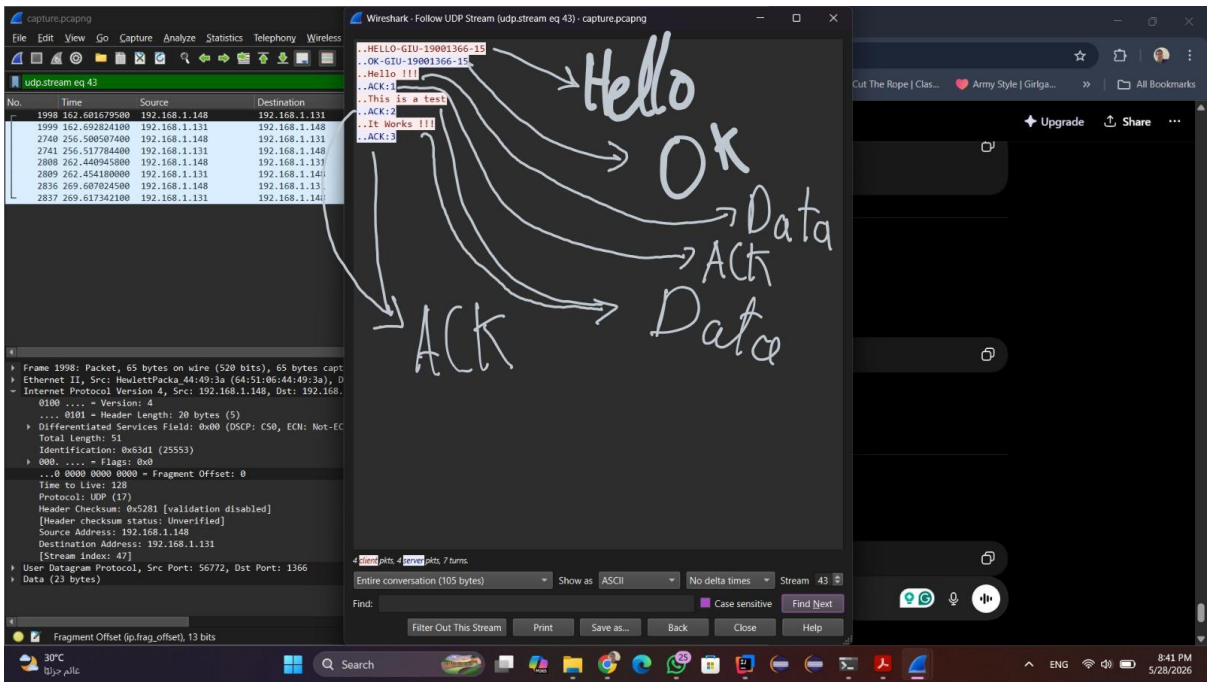
| Network | Mask | Next Hop | Interface |
|---|---|---|---|
| 19.0.13.0 | 255.255.255.128 | directly connected | Fa0/0 |
| 19.0.13.128 | 255.255.255.192 | directly connected | Fa0/1 |
| 19.0.13.192 | 255.255.255.240 | directly connected | Fa1/0 |
| 19.0.13.208 | 255.255.255.252 | directly connected | Se0/3/0 |
| 19.0.12.0 | 255.255.255.0 | 19.0.13.210 | Se0/3/0 (static) |

## 5.2 Router 2 (Router1) Routing Table

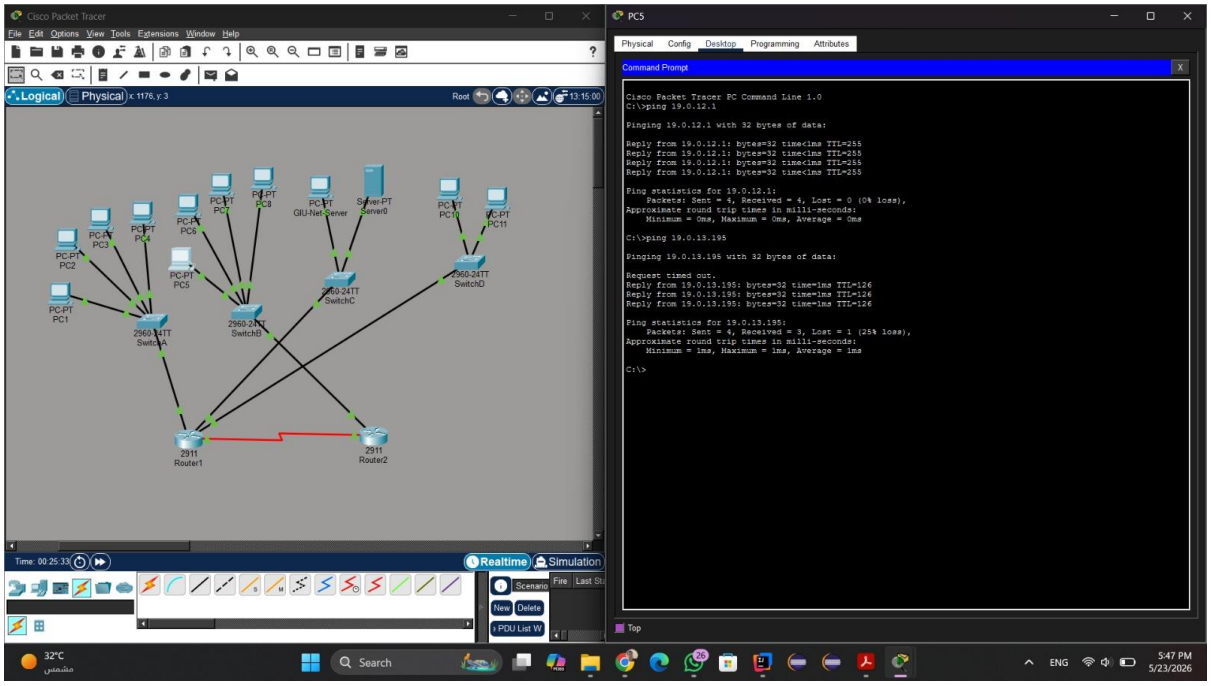
| Network | Mask | Next Hop | Interface |
|---|---|---|---|
| 19.0.12.0 | 255.255.255.0 | directly connected | Fa0/0 |
| 19.0.13.208 | 255.255.255.252 | directly connected | Se0/3/0 |
| 19.0.13.0 | 255.255.255.128 | 19.0.13.209 | Se0/3/0 (static) |
| 19.0.13.128 | 255.255.255.192 | 19.0.13.209 | Se0/3/0 (static) |
| 19.0.13.192 | 255.255.255.240 | 19.0.13.209 | Se0/3/0 (static) |

# 6. Screenshots

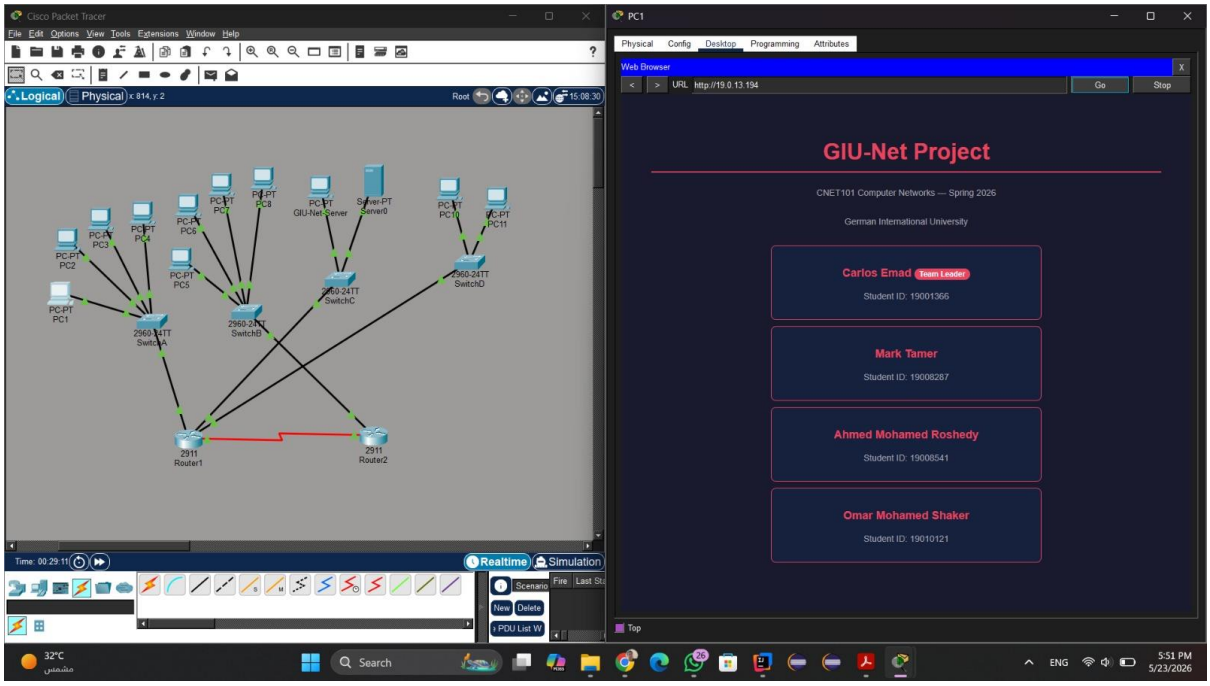
## 6.1 Wireshark Capture



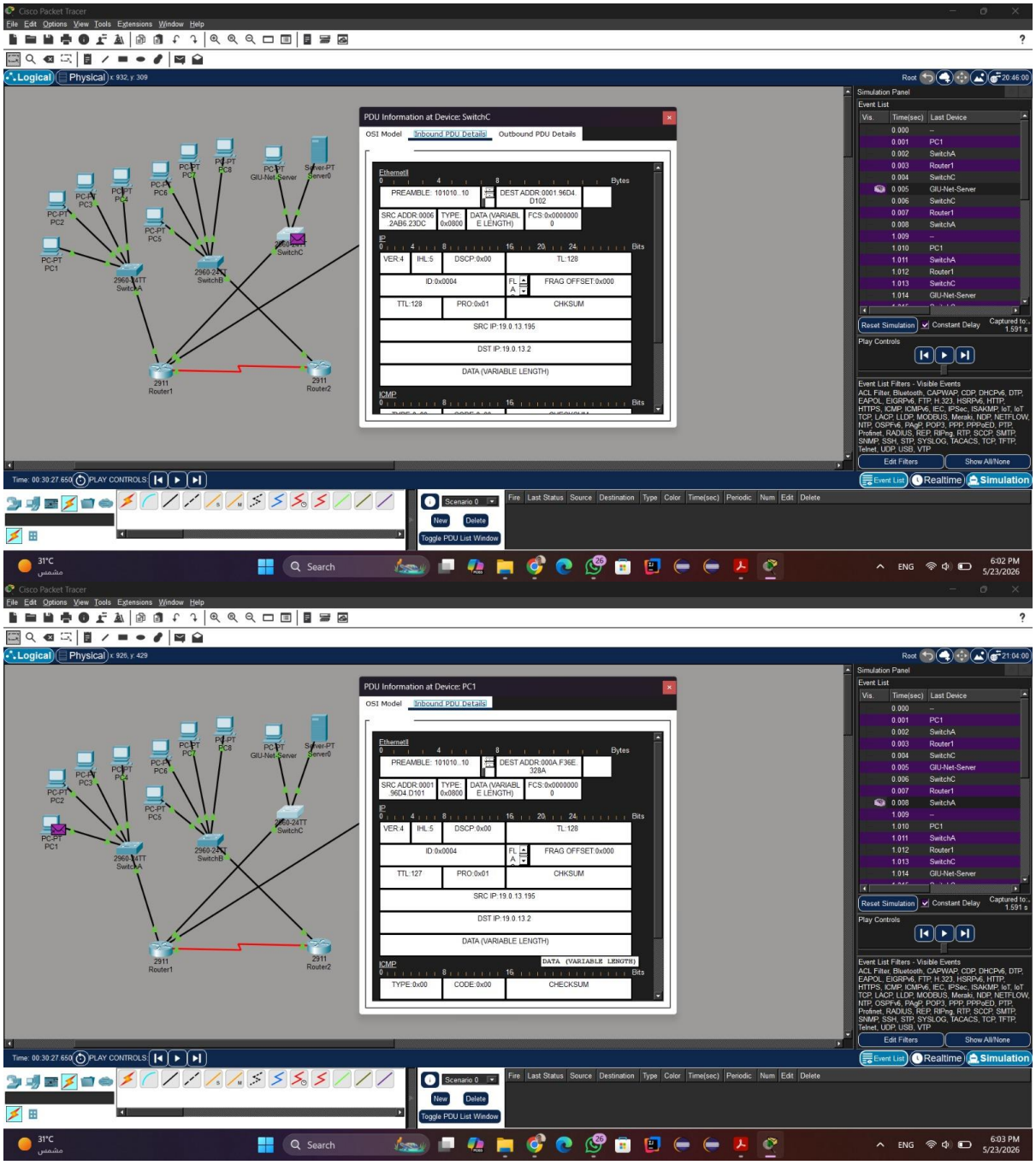

## 6.2 Successful Ping — Housing PC to GIU-Net Server

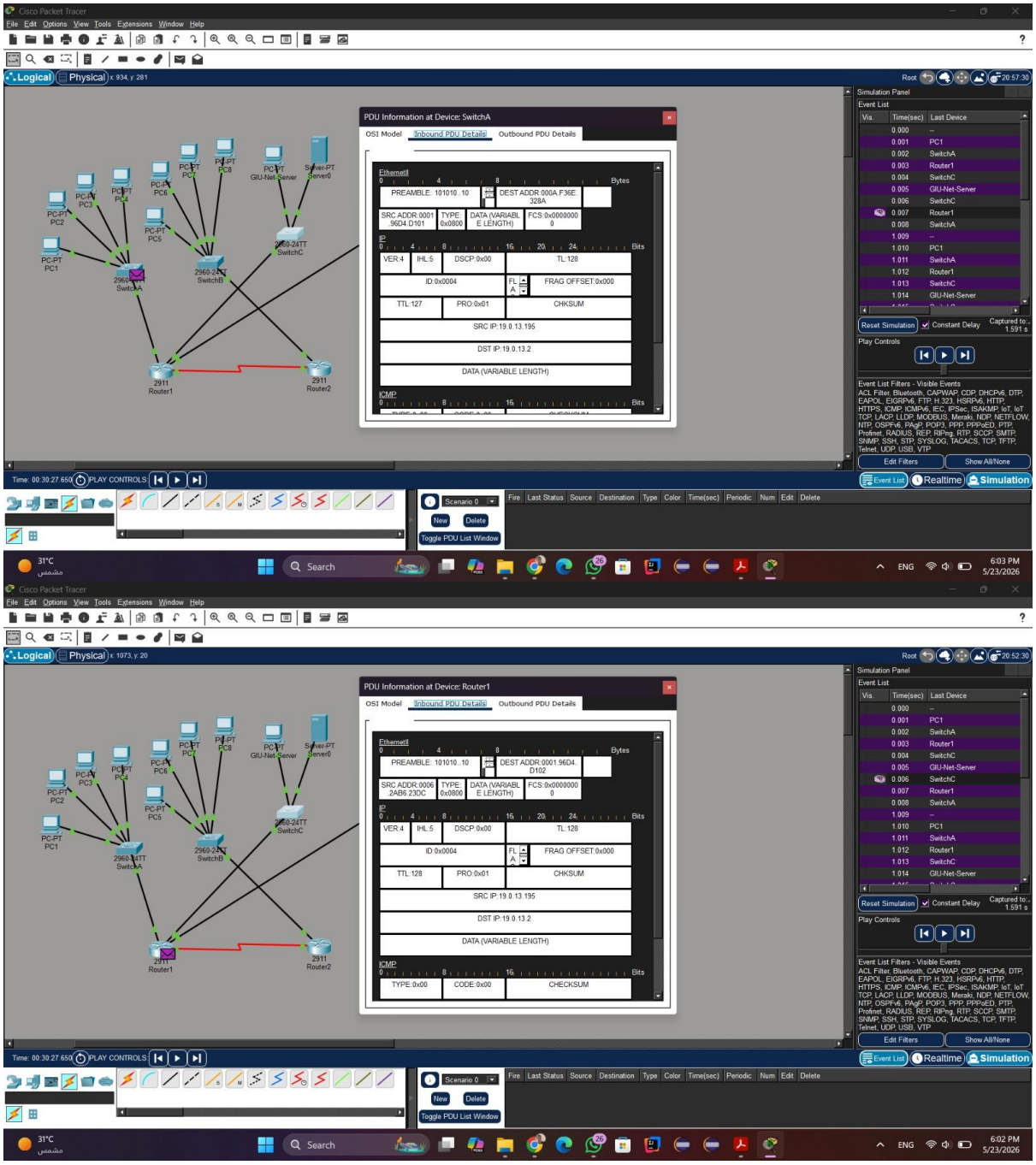


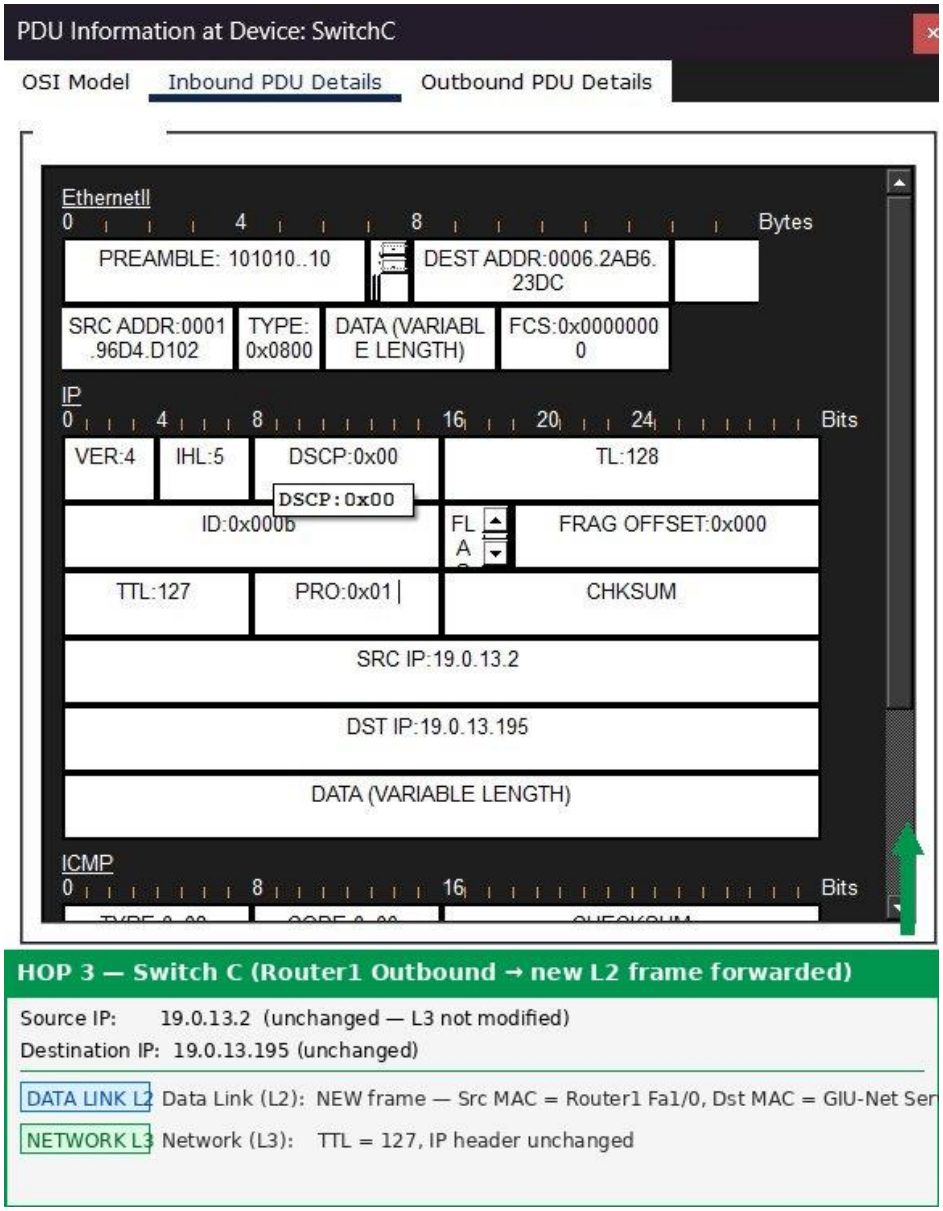

## 6.3 HTTP Webpage




## 6.4 Simulation Mode Packet Trace


Cisco Packet Tracer
File Edit Options View Tools Extensions Window Help
Logical Physical
PDU Information at Device: SwitchC
OSI Model Inbound PDU Details Outbound PDU Details
Ethernet II
PREAMBLE: 101010..10
DEST ADDR:0001.96D4.D102
SRC ADDR:0006.2AB6.23DC
TYPE: 0x0800
DATA (VARIABLE LENGTH)
FCS:0x00000000
IP
VER:4
IHL:5
DSCP:0x00
TL:128
ID:0x0004
FRAG OFFSET:0x000
TTL:128
PRO:0x01
CHKSUM
SRC IP:19.0.13.195
DST IP:19.0.13.2
DATA (VARIABLE LENGTH)
ICMP
Simulation Panel
Event List
Reset Simulation
Constant Delay
Play Controls
Edit Filters
Show All/None
Realtime
Simulation
Scenario 0
New
Delete
Toggle PDU List Window
Search
6:02 PM
5/23/2026
Cisco Packet Tracer
PDU Information at Device: PC1
OSI Model Inbound PDU Details
Ethernet II
PREAMBLE: 101010..10
DEST ADDR:000A.F36E.329A
SRC ADDR:0001.96D4.D101
TYPE: 0x0800
DATA (VARIABLE LENGTH)
FCS:0x00000000
IP
VER:4
IHL:5
DSCP:0x00
TL:128
ID:0x0004
FRAG OFFSET:0x000
TTL:127
PRO:0x01
CHKSUM
SRC IP:19.0.13.195
DST IP:19.0.13.2
DATA (VARIABLE LENGTH)
ICMP
DATA (VARIABLE LENGTH)
TYPE:0x00
CODE:0x00
CHECKSUM
Simulation Panel
Event List
Reset Simulation
Constant Delay
Play Controls
Edit Filters
Show All/None
Realtime
Simulation
Scenario 0
New
Delete
Toggle PDU List Window
Search
6:03 PM
5/23/2026


Cisco Packet Tracer
File Edit Options View Tools Extensions Window Help
Logical Physical
PDU Information at Device: SwitchA
OSI Model Inbound PDU Details Outbound PDU Details
Ethernet II
PREAMBLE: 101010..10
DEST ADDR:000A.F36E.328A
SRC ADDR:0001.96D4.D101
TYPE: 0x0800
DATA (VARIABLE LENGTH)
FCS:0x00000000
IP
VER:4
IHL:5
DSCP:0x00
TL:128
ID:0x0004
FRAG OFFSET:0x000
TTL:127
PRO:0x01
CHKSUM
SRC IP:19.0.13.195
DST IP:19.0.13.2
DATA (VARIABLE LENGTH)
ICMP
Simulation Panel
Event List
Reset Simulation
Constant Delay
Play Controls
Edit Filters
Show All/None
Realtime
Simulation
Scenario 0
New
Delete
Toggle PDU List Window
PDU Information at Device: Router1
OSI Model Inbound PDU Details Outbound PDU Details
Ethernet II
PREAMBLE: 101010..10
DEST ADDR:0001.96D4.D102
SRC ADDR:0006.2AB6.23DC
TYPE: 0x0800
DATA (VARIABLE LENGTH)
FCS:0x00000000
IP
VER:4
IHL:5
DSCP:0x00
TL:128
ID:0x0004
FRAG OFFSET:0x000
TTL:128
PRO:0x01
CHKSUM
SRC IP:19.0.13.195
DST IP:19.0.13.2
DATA (VARIABLE LENGTH)
ICMP
TYPE:0x00
CODE:0x00
CHECKSUM
Simulation Panel
Event List
Reset Simulation
Constant Delay
Play Controls
Edit Filters
Show All/None
Realtime
Simulation
Scenario 0
New
Delete
Toggle PDU List Window



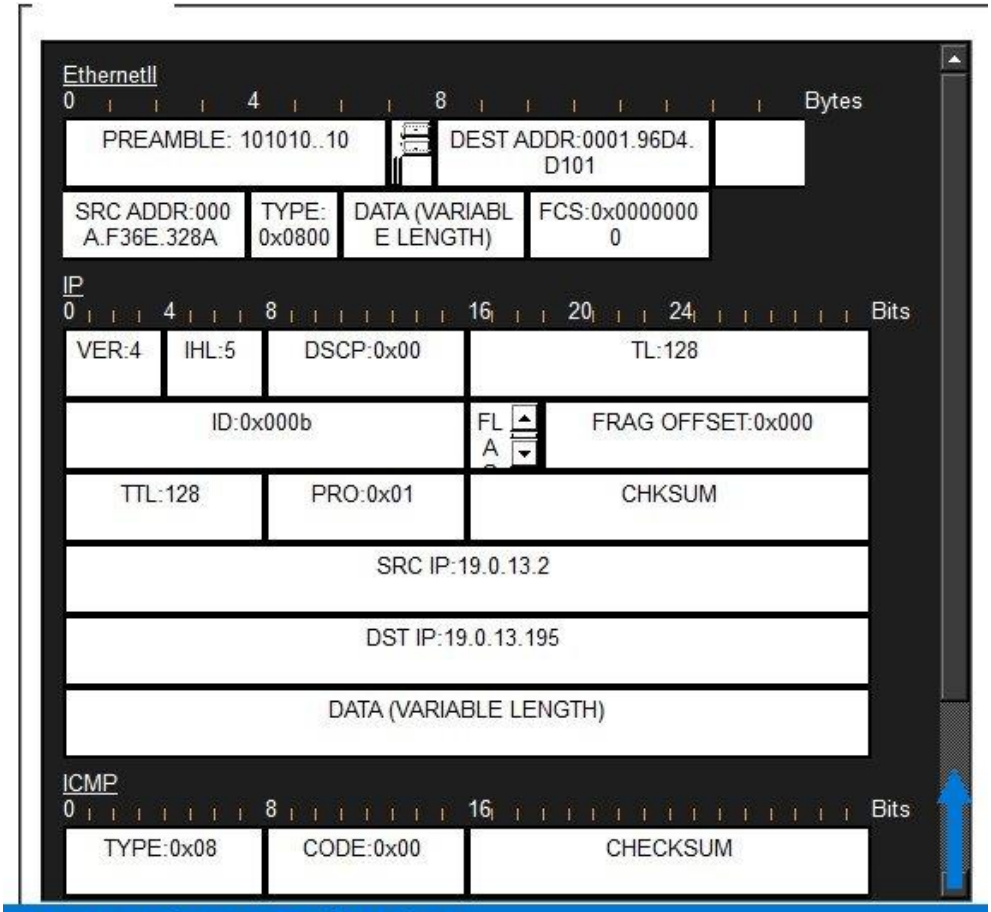

**HOP 3 — Switch C (Router1 Outbound → new L2 frame forwarded)**

Source IP: 19.0.13.2 (unchanged — L3 not modified)

Destination IP: 19.0.13.195 (unchanged)

DATA LINK L2 Data Link (L2): NEW frame — Src MAC = Router1 Fa1/0, Dst MAC = GIU-Net Ser

NETWORK L3 Network (L3): TTL = 127, IP header unchanged



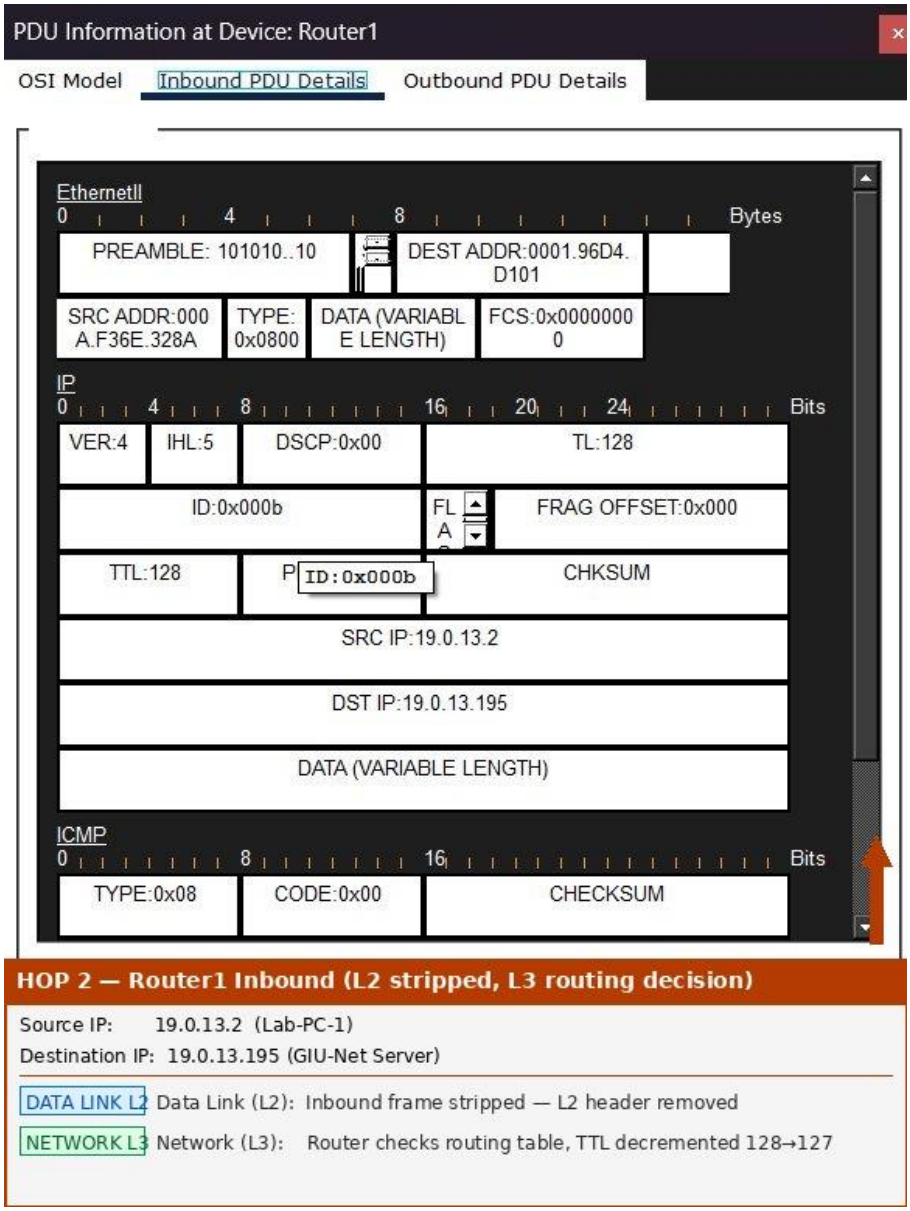

**HOP 1 — Lab-PC-1 Sends Packet**

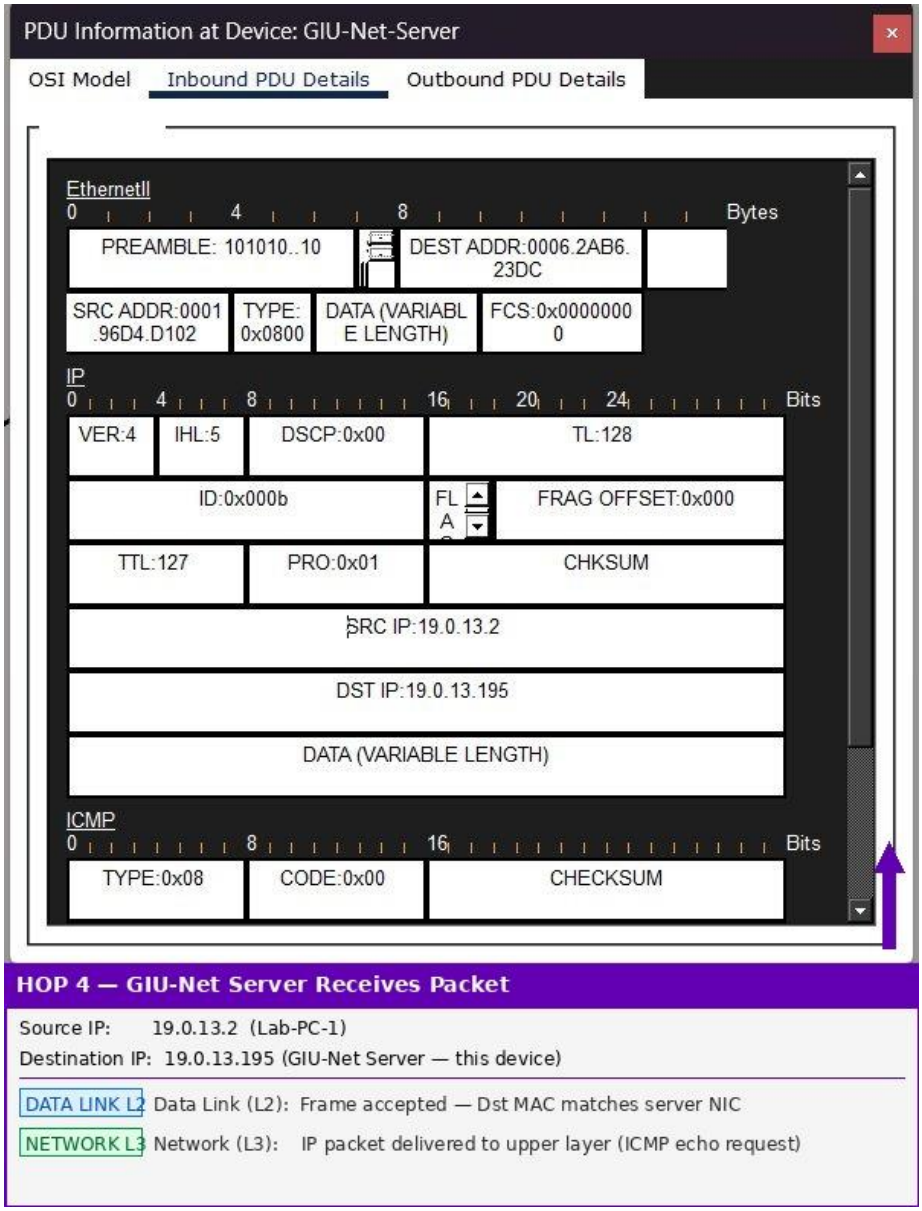
Source IP: 19.0.13.2 (Lab-PC-1)

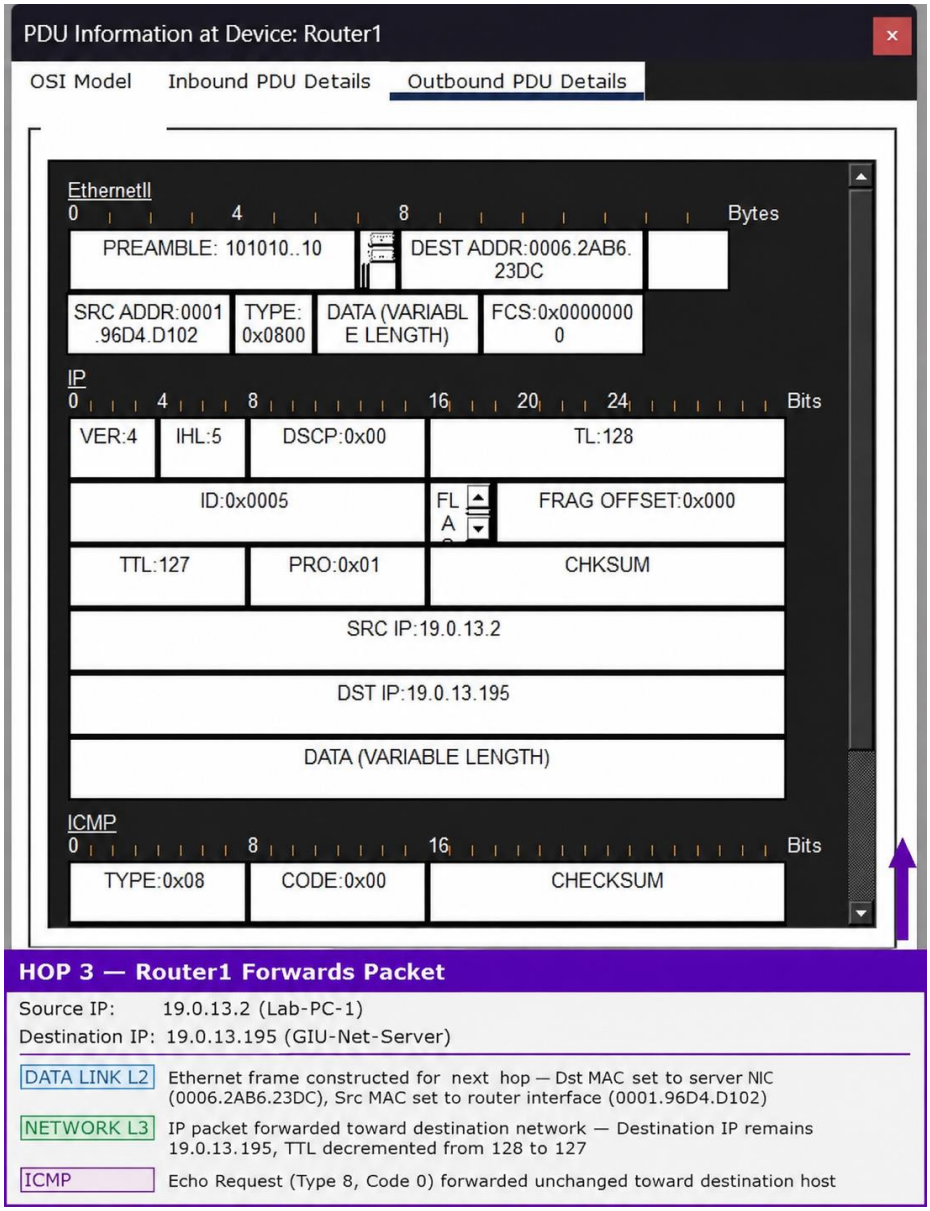
Destination IP: 19.0.13.195 (GIU-Net Server)

DATA LINK L2 Data Link (L2): Frame built — Src MAC = Lab-PC-1, Dst MAC = Router1

NETWORK L3 Network (L3): IP packet created, TTL = 128

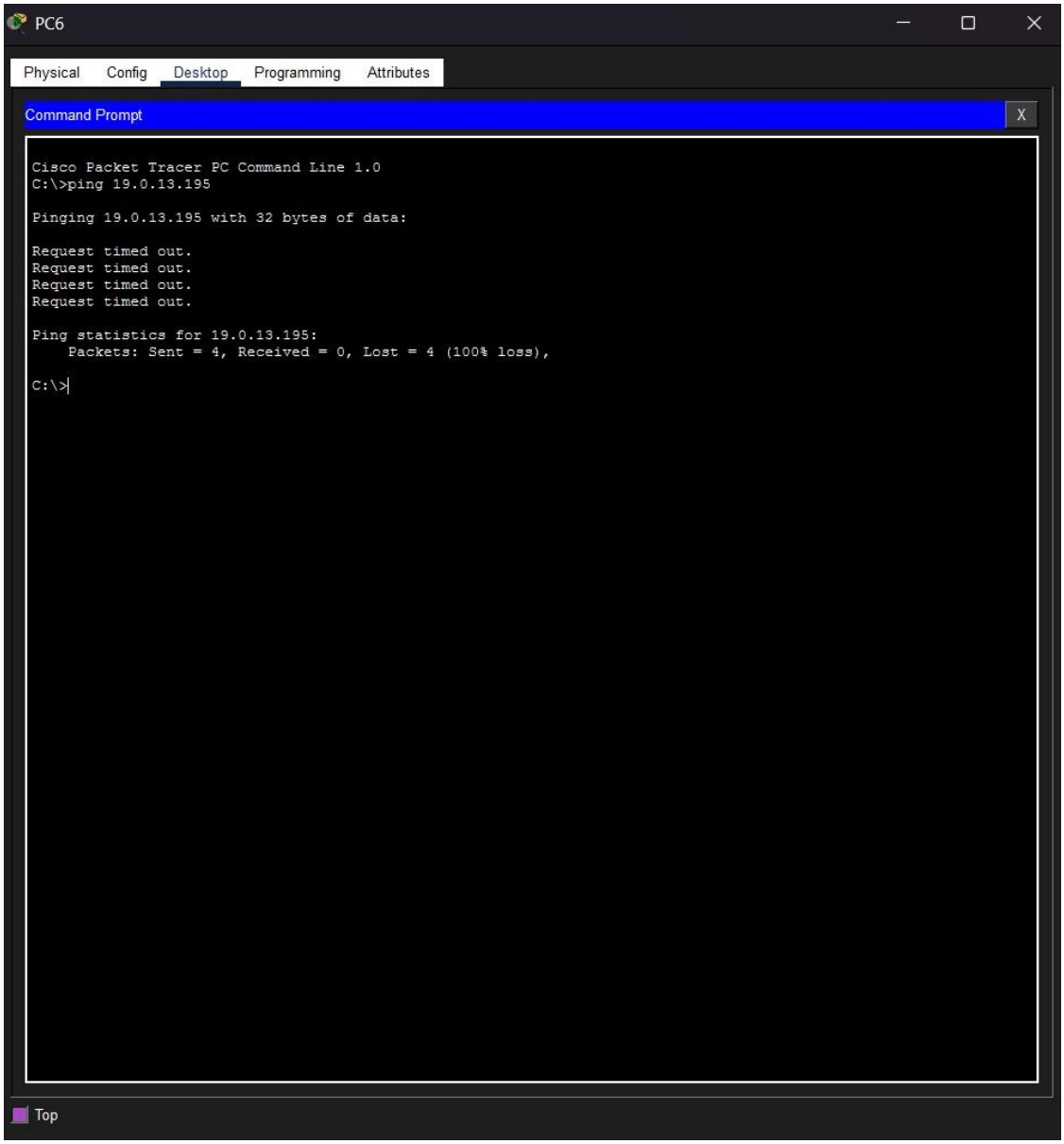

PDU Information at Device: Router1
OSI Model
Inbound PDU Details
Outbound PDU Details
EthernetII
0
4
8
Bytes
PREAMBLE: 101010..10
DEST ADDR:0001.96D4.D101
SRC ADDR:000A.F36E.328A
TYPE: 0x0800
DATA (VARIABLE LENGTH)
FCS:0x00000000
IP
0
4
8
16
20
24
Bits
VER:4
IHL:5
DSCP:0x00
TL:128
ID:0x000b
FRAG OFFSET:0x000
TTL:128
CHKSUM
SRC IP:19.0.13.2
DST IP:19.0.13.195
DATA (VARIABLE LENGTH)
ICMP
0
8
16
Bits
TYPE:0x08
CODE:0x00
CHECKSUM
HOP 2 — Router1 Inbound (L2 stripped, L3 routing decision)
Source IP: 19.0.13.2 (Lab-PC-1)
Destination IP: 19.0.13.195 (GIU-Net Server)
DATA LINK L2 Data Link (L2): Inbound frame stripped — L2 header removed
NETWORK L3 Network (L3): Router checks routing table, TTL decremented 128→127


PDU Information at Device: GIU-Net-Server
OSI Model
Inbound PDU Details
Outbound PDU Details
EthernetII
0
4
8
Bytes
PREAMBLE: 101010..10
DEST ADDR:0006.2AB6.23DC
SRC ADDR:0001.96D4.D102
TYPE: 0x0800
DATA (VARIABLE LENGTH)
FCS:0x00000000
IP
0
4
8
16
20
24
Bits
VER:4
IHL:5
DSCP:0x00
TL:128
ID:0x000b
FRAG OFFSET:0x000
TTL:127
PRO:0x01
CHKSUM
SRC IP:19.0.13.2
DST IP:19.0.13.195
DATA (VARIABLE LENGTH)
ICMP
0
8
16
Bits
TYPE:0x08
CODE:0x00
CHECKSUM
HOP 4 — GIU-Net Server Receives Packet
Source IP: 19.0.13.2 (Lab-PC-1)
Destination IP: 19.0.13.195 (GIU-Net Server — this device)
DATA LINK L2
Data Link (L2): Frame accepted — Dst MAC matches server NIC
NETWORK L3
Network (L3): IP packet delivered to upper layer (ICMP echo request)




## 6.5 ACL Demonstration – Housing PC Cannot reach GIU-Net Server

To verify the ACL is enforcing the deny policy, a ping was issued from Housing-PC-1 (Subnet C, 19.0.12.x) to the GIU-Net Server (Subnet E, 19.0.13.195). As expected, all four packets timed out, confirming that Router 2's ACL 101 is dropping traffic from Subnet C before it reaches the WAN link.




## 6.6 ACL Demonstration – Lab PC Can Reach GIU-Net Server

To confirm that the ACL does not affect other traffic, a ping was issued from Lab-PC-1 (Subnet B, 19.0.13.x) to the GIU-Net Server (19.0.13.195). All four packets received replies. This is expected because Lab traffic travels entirely through Router 1 and never passes through Router 2 or its ACL.

# 7. Bonus – Task 5: Access Control List

## 7.1 ACL Configuration

An extended ACL was configured on Router 2 (named Router1 in Cisco Packet Tracer) to deny all IP traffic originating from Subnet C (Student Housing, 19.0.12.0/24) and destined for Subnet E (Server DMZ, 19.0.13.192/28), while permitting all other traffic.

The complete IOS configuration applied is as follows:

```
! Extended ACL 101 — deny Housing-to-DMZ, permit everything else
access-list 101 deny   ip 19.0.12.0 0.0.0.255 19.0.13.192 0.0.0.15
access-list 101 permit ip any any

! Apply the ACL inbound on Fa0/0 (the Housing-facing interface of Router 2)
interface FastEthernet0/0
ip access-group 101 in
```

**Wildcard mask derivation:**
The wildcard mask is the bitwise inverse of the subnet mask.
For Subnet C (/24): subnet mask = 255.255.255.0, so wildcard = 0.0.0.255. This matches any host in the 19.0.12.0 to 19.0.12.255 range.
For Subnet E (/28): subnet mask = 255.255.255.240, so wildcard = 0.0.0.15. This matches any host in the 19.0.13.192 to 19.0.13.207 range.

**Why the ACL is placed inbound on Router 2's Fa0/0:**
Extended ACLs are placed as close to the traffic source as possible. All Housing PC traffic enters Router 2 through Fa0/0. Filtering at this point drops the packet immediately, before it consumes any WAN bandwidth on Subnet D. Placing the ACL on the outbound Serial interface or on Router 1 would still work functionally, but would waste resources forwarding packets that will ultimately be discarded. Lab PC traffic (Subnet B) is routed entirely through Router 1 and never traverses Router 2, so it is unaffected by this ACL regardless of placement.

**Verification output:**
The following commands confirm the ACL is correctly defined and applied:

```
Router1# show ip access-lists
Extended IP access list 101
    10 deny ip 19.0.12.0 0.0.0.255 19.0.13.192 0.0.0.15
    20 permit ip any any

Router1# show ip interface FastEthernet0/0
  Inbound access list is 101
```

## 7.2 ACL vs Application-Layer Handshake – Access Control at Two Layers

Both the ACL configured on Router 2 and the HELLO/REJECTED handshake implemented in GIUServer.java enforce access control, but they operate at completely different layers of the network stack.

The ACL operates at the **Network Layer (Layer 3)**. It examines the source and destination IP addresses of every packet entering Router 2's Fa0/0 interface and silently drops any packet from Subnet C (19.0.12.0/24) destined for Subnet E (19.0.13.192/28). This happens in the network infrastructure, entirely transparently to the application. The GIU-Net Server receives no indication that the packet was ever sent — it never arrives at the server's socket. The ACL does not know or care what application is running; it enforces policy purely based on IP addresses.

The HELLO/REJECTED handshake operates at the **Application Layer (Layer 7)**. By the time the server evaluates a HELLO packet, that packet has already successfully traversed all routers, crossed subnet boundaries, and been received by the server's DatagramSocket. Only then does the server inspect the payload string, compute the expected HELLO-GIU-19001366-15 value, and decide whether to accept the connection with OK or reject it with REJECTED. This mechanism enforces policy based on application-level credentials — the team's leader ID and digit sum — rather than on network addresses.

The two mechanisms are complementary. The ACL acts as a coarse network-level gate: it can block entire subnets from even reaching the server. The handshake acts as a fine application-level gate: it ensures that even a host which does reach the server must still present the correct credential. A real-world DMZ deployment would use both — the network layer restricts which machines can communicate with the server at all, while the application layer authenticates the individual session. This is the principle of **defence in depth**: access control enforced redundantly at multiple layers of the stack, so that a failure or bypass at one layer does not immediately compromise the entire system.

The Packet Tracer campus network represents the physical and logical infrastructure that would carry GIU-Net traffic in real deployment. The two parts of this project are directly related:

- GIUServer.java runs on the GIU-Net Server device (Subnet E, 19.0.13.195), listening on UDP port 1366.
- GIUClient.java runs on a PC in any subnet (e.g., Lab, Subnet B) and sends UDP packets destined for 19.0.13.195
- :1366.
- The static routes configured on Router 1 and Router 2 ensure packets are forwarded correctly between subnets.
- The HELLO/REJECTED handshake in the Java code enforces access control at the application layer, just as the ACL in Task 5 enforces it at the network layer — demonstrating how security is implemented at multiple layers of the network stack.